

Elixir style: source comparison

This comparison is produced by the `green_validation` project, a companion to `green`, the Elixir code formatter plugin. This document compares the four most widely used sources of truth for Elixir code style (the mix format formatter, Lexmag's style guide, Credo's style guide, and christopheradams' "The Elixir Style Guide") across 114 catalogued rules, showing which sources propose each one. Generated with Elixir 1.19.5. (version 27dae67, generated on 2026-07-01).

How to read it: each column is a style guide, and "Yes" marks where that guide proposes the rule. Rules are grouped first into those mix format enforces (shown with their category) and then by category. Numbered footnotes record parameter-level differences between sources, such as the maximum line length.

Sources

mix format = mix format: <https://github.com/elixir-lang/elixir>

Lexmag = Lexmag's Elixir Style Guide: <https://github.com/lexmag/elixir-style-guide>

Credo = Credo's Elixir Style Guide: <https://github.com/rrrene/elixir-style-guide>

Adams = The Elixir Style Guide: https://github.com/christopheradams/elixir_style_guide

Enforced by mix format

Category	Rule	Lexmag	Credo	Adams
Expressions	Format with/else over multiple lines			Yes
Expressions	No parentheses around anonymous function arguments	Yes		
Expressions	No redundant parentheses			
Expressions	Omit brackets from keyword lists			Yes
Expressions	Parentheses around definition arguments	Yes	Yes	
Expressions	Parentheses on function calls with arguments		Yes	Yes
Expressions	Parentheses on piped one-arity calls			Yes
Expressions	Parentheses on zero-arity function calls	Yes		Yes
Formatting	Avoid aligning expression groups	Yes		
Formatting	Binary operators at end of line	Yes		
Formatting	Blank line after a multi-line assignment			Yes
Formatting	Blank lines around multi-line case clauses			Yes
Formatting	Capture operator spacing			
Formatting	Collapse consecutive blank lines			

Category	Rule	Lexmag	Credo	Adams
Formatting	Comments on their own line			Yes
Formatting	Fit collections on a single line when they fit			
Formatting	for clause indentation	Yes		
Formatting	Group digits in large numbers	Yes	Yes	
Formatting	Guard clause indentation	Yes		
Formatting	Indent multi-line binary operands	Yes		
Formatting	Lowercase exponent letter in floats			
Formatting	Maximum line length [1]		Yes [2]	Yes [3]
Formatting	Multi-line expression on its own line after =	Yes	Yes	
Formatting	No blank line after defmodule			Yes
Formatting	No semicolons between statements	Yes	Yes	
Formatting	No space after the ! operator	Yes	Yes	Yes
Formatting	No space between a function name and (			Yes
Formatting	No spaces around bitstring segment options	Yes		

Category	Rule	Lexmag	Credo	Adams
Formatting	No spaces around the range operator	Yes		Yes
Formatting	No spaces inside brackets		Yes	Yes
Formatting	No trailing comma in collections	Yes		
Formatting	No trailing whitespace	Yes	Yes	Yes
Formatting	One element per line in multi-line collections			Yes
Formatting	Quote atoms only when required	Yes		
Formatting	Single-level pipeline indentation	Yes		
Formatting	Space after commas	Yes	Yes	Yes
Formatting	Space after the comment marker	Yes		Yes
Formatting	Space before -> in 0-arity anonymous functions	Yes		
Formatting	Spaces around binary operators	Yes	Yes	Yes
Formatting	Spaces around the -> operator			
Formatting	Spaces around the default-argument operator	Yes		
Formatting	Trailing newline at end of file	Yes	Yes	Yes

Category	Rule	Lexmag	Credo	Adams
Formatting	Two-space indentation	Yes	Yes	
Formatting	Unix line endings		Yes	Yes
Formatting	Uppercase hexadecimal literals	Yes		
Formatting	with clause indentation	Yes		Yes
Modules	Omit brackets from a defstruct keyword list			Yes
Typespecs	One part per line in long union types			Yes

Formatting

Rule	Lexmag	Credo	Adams
Long do: clause on its own line			Yes
Opening bracket stays on the assignment line			Yes
Use vertical space between definitions		Yes	Yes

Naming

Rule	Lexmag	Credo	Adams
Avoid one-letter variable names	Yes		
CamelCase for modules	Yes	Yes	Yes
Don't shadow Kernel functions		Yes	
is_ prefix for guard-safe predicates			Yes
Private function names differ from public			Yes
Question mark for predicate functions	Yes	Yes	Yes
snake_case for atoms, variables and functions	Yes	Yes	Yes
snake_case for files and directories	Yes		Yes

Modules

Rule	Lexmag	Credo	Adams
Alias a long module self-reference			Yes
Alias the modules you use		Yes	
Avoid repetitive module names			Yes
Consistent def line style			Yes
Consistent module attribute layout	Yes [4]		Yes [5]
Don't shadow stdlib modules		Yes	
Group clauses of the same function		Yes	Yes
Module name mirrors its file path			Yes
No explicit nil struct field defaults	Yes		Yes
One module per file			Yes
Order alias/import/require alphabetically			Yes
Use the __MODULE__ pseudo-variable	Yes	Yes	Yes

Typespecs

Rule	Lexmag	Credo	Adams
@spec directly before the function			Yes
Keep @typedoc with its @type			Yes
Name the main type t			Yes
Parentheses on zero-arity types	Yes		

Expressions

Rule	Lexmag	Credo	Adams
Avoid needless metaprogramming			Yes
Avoid needless pipelines	Yes		Yes
Don't deeply nest conditionals		Yes	
No always-true conditionals		Yes	
No anonymous functions in pipelines	Yes		
No else clause with unless	Yes	Yes	Yes
No IEx.pry in production code		Yes	
No IO.inspect in production code		Yes	
No parentheses around if/unless conditions		Yes	
No unless with a negated condition		Yes	
Omit else that returns nil	Yes		
Omit parentheses on zero-argument definitions			Yes
Shorthand map syntax for atom keys			Yes
Start pipe chains with a raw value		Yes	Yes

Rule	Lexmag	Credo	Adams
Use <> to match binaries	Yes		Yes
Use and/or/not for boolean checks	Yes		
Use do: for single-line if/unless			Yes
Use keyword-list shorthand			Yes
Use pipelines for chained calls	Yes		Yes
Use true as the last cond clause	Yes		Yes
Verbose map syntax for non-atom keys			Yes

Documentation

Rule	Lexmag	Credo	Adams
@moduledoc false when undocumented		Yes	Yes
Always include @moduledoc			Yes
Blank line after @moduledoc		Yes	Yes
Use heredocs for documentation			Yes

Comments

Rule	Lexmag	Credo	Adams
Avoid superfluous comments	Yes		
Capitalise and punctuate comments			Yes
Comments only for important details	Yes		
Limit comment lines to 100 characters			Yes
Prefer @doc/@moduledoc over comments		Yes	
Uppercase annotation keywords (TODO/FIXME/âf)		Yes	Yes

Testing

Rule	Lexmag	Credo	Adams
Expected value on the right in assertions	Yes		Yes

Exceptions

Rule	Lexmag	Credo	Adams
Error suffix for exception modules	Yes	Yes	Yes
Lowercase exception messages	Yes		Yes
No trailing punctuation in exception messages	Yes		Yes

Regex

Rule	Lexmag	Credo	Adams
Prefer pattern matching over regular expressions	Yes		
Use \A and \Z for string boundaries	Yes	Yes	
Use non-capturing regex groups	Yes		
Use the ~r sigil for regexes		Yes	

Notes

[1] Maximum line length (mix format): 98-column default

[2] Maximum line length (Credo): keep lines under 80 characters

[3] Maximum line length (Adams): limit lines to 98 characters

[4] Consistent module attribute layout (Lexmag): order: use, import, alias, require

[5] Consistent module attribute layout (Adams): order: use, import, require, alias

Examples

Formatting

Avoid aligning expression groups

```
# bad
a   = 1
bbb = 2

# good
a = 1
bbb = 2
```

Binary operators at end of line

```
# bad
value = aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa and bbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbb and ccccccccccccccccccccccccccccccccccc

# good
value =
    aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa and bbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbb and
    ccccccccccccccccccccccccccccccccccc
```

Blank line after a multi-line assignment

```
# bad
x =
    foo()
    |> bar()
y = 1

# good
x =
    foo()
    |> bar()

y = 1
```

Blank lines around multi-line case clauses

```
# bad
case x do
  1 -> a
  2 ->
    b
end

# good
case x do
  1 ->
    a

  2 ->
    b
end
```

Capture operator spacing

```
# bad
&(&1)

# good
& &1
```

Collapse consecutive blank lines

```
# bad
a = 1

b = 2

# good
a = 1

b = 2
```

Comments on their own line

```
# bad
x = 1 # set x

# good
# set x
x = 1
```

Fit collections on a single line when they fit

```
# bad
[1,
2]

# good
[1, 2]
```

for clause indentation

```
# bad
for a <- as,
  b <- bs do
  {a, b}
end

# good
for a <- as,
  b <- bs do
  {a, b}
end
```

Group digits in large numbers

```
# bad
10000000

# good
1_000_000
```

Guard clause indentation

```
# bad
def f(x)
when x > 0 do
  x
end

# good
def f(x)
  when x > 0 do
    x
  end
end
```

Indent multi-line binary operands

```
# bad
x =
  foo() +
  bar()

# good
x =
  foo() +
  bar()
```


Long do: clause on its own line

```
# bad
def handle(arg), do: a_rather_long_expression_that_overflows(arg)

# good
def handle(arg),
  do: a_rather_long_expression_that_overflows(arg)
```

Lowercase exponent letter in floats

```
# bad
1.0E3

# good
1.0e3
```

Maximum line length

```
# bad
result = some_function(argument_one, argument_two, argument_three, argument_four, argument_five, argument_six)

# good
result =
  some_function(
    argument_one,
    argument_two,
    argument_three,
    argument_four,
    argument_five,
    argument_six
  )
```

Multi-line expression on its own line after =

```
# bad
x = foo()
    |> bar()

# good
x =
  foo()
  |> bar()
```

No blank line after defmodule

```
# bad
defmodule A do

  def x, do: 1
end

# good
defmodule A do
  def x, do: 1
end
```

No semicolons between statements

```
# bad
a = 1; b = 2

# good
a = 1
b = 2
```

No space after the ! operator

```
# bad
! ready

# good
!ready
```

No space between a function name and (

```
# bad
foo (x)

# good
foo(x)
```

No spaces around bitstring segment options

```
# bad
<<1 ::8>>

# good
<<1::8>>
```

No spaces around the range operator

```
# bad
1 .. 2

# good
1..2
```

No spaces inside brackets

```
# bad
[ 1, 2 ]

# good
[1, 2]
```

No trailing comma in collections

```
# bad
[1, 2, ]

# good
[1, 2]
```

No trailing whitespace

```
# bad
x = 1

# good
x = 1
```

One element per line in multi-line collections

```
# bad
[
  1, 2,
  3
]

# good
[
  1,
  2,
  3
]
```

Opening bracket stays on the assignment line

```
# bad
x =
  [
    1,
    2
  ]

# good
x = [
  1,
  2
]
```

Quote atoms only when required

```
# bad
:"valid"

# good
:valid
```

Single-level pipeline indentation

```
# bad
x
  |> f()
  |> g()

# good
x
|> f()
|> g()
```

Space after commas

```
# bad
[1,2]

# good
[1, 2]
```

Space after the comment marker

```
# bad
#comment

# good
# comment
```

Space before -> in 0-arity anonymous functions

```
# bad
fn-> :ok end

# good
fn -> :ok end
```

Spaces around binary operators

```
# bad
1+1

# good
1 + 1
```

Spaces around the -> operator

```
# bad
fn x->x end

# good
fn x -> x end
```

Spaces around the default-argument operator

```
# bad
def f(x \\0), do: x

# good
def f(x \\ 0), do: x
```

Trailing newline at end of file

```
# bad
(file ends without a newline)

# good
(file ends with one \n)
```

Two-space indentation

```
# bad
if x do
  :ok
end

# good
if x do
  :ok
end
```

Unix line endings

```
# bad
a = 1\r\nb = 2

# good
a = 1\nb = 2
```

Uppercase hexadecimal literals

```
# bad
0xabcd

# good
0xABCD
```

Use vertical space between definitions

```
# bad
def a, do: 1
def b, do: 2

# good
def a, do: 1

def b, do: 2
```


with clause indentation

```
# bad
with {:ok, a} <- f(),
  {:ok, b} <- g() do
  a + b
end

# good
with {:ok, a} <- f(),
     {:ok, b} <- g() do
  a + b
end
```

Naming

Avoid one-letter variable names

```
# bad
u = get_user()

# good
user = get_user()
```

CamelCase for modules

```
# bad
defmodule My_app do
end

# good
defmodule MyApp do
end
```

Don't shadow Kernel functions

```
# bad
def length(list), do: count(list)

# good
def list_length(list), do: count(list)
```

is_ prefix for guard-safe predicates

```
# bad
defguard valid?(x) when x > 0

# good
defguard is_valid(x) when x > 0
```

Private function names differ from public

```
# bad
def f(x), do: x
defp f(x, y), do: x + y

# good
def f(x), do: x
defp do_f(x, y), do: x + y
```

Question mark for predicate functions

```
# bad
def is_valid(x), do: true

# good
def valid?(x), do: true
```

snake_case for atoms, variables and functions

```
# bad
fooBar = 1

# good
foo_bar = 1
```

snake_case for files and directories

```
# bad
lib/MyApp/UserProfile.ex

# good
lib/my_app/user_profile.ex
```

Modules

Alias a long module self-reference

```
# bad
%MyApp.Deeply.Nested.Thing{}

# good
alias __MODULE__, as: Thing
%Thing{}
```

Alias the modules you use

```
# bad
MyApp.Accounts.User.changeset(u)

# good
alias MyApp.Accounts.User
User.changeset(u)
```

Avoid repetitive module names

```
# bad
MyApp.User.UserHelper

# good
MyApp.User.Helper
```

Consistent def line style

```
# bad
def f(0), do: 0

def f(n) do
  n + 1
end

# good
def f(0) do
  0
end

def f(n) do
  n + 1
end
```

Consistent module attribute layout

```
# bad
alias App.X
use GenServer
@moduledoc "..."/>
```

Don't shadow stdlib modules

```
# bad
defmodule String do
end

# good
defmodule MyApp.Text do
end
```

Group clauses of the same function

```
# bad
def f(1), do: 1
def g(), do: 0
def f(2), do: 2

# good
def f(1), do: 1
def f(2), do: 2

def g(), do: 0
```

Module name mirrors its file path

```
# bad
lib/parser.ex defining MyApp.Lexer.Parser

# good
lib/my_app/lexer/parser.ex defining MyApp.Lexer.Parser
```

No explicit nil struct field defaults

```
# bad
defstruct name: nil, age: nil

# good
defstruct [:name, :age]
```

Omit brackets from a defstruct keyword list

```
# bad
defstruct [name: nil, age: 0]

# good
defstruct name: nil, age: 0
```

One module per file

```
# bad
defmodule A do
end

defmodule B do
end

# good
# A and B in separate files
```

Order alias/import/require alphabetically

```
# bad
alias App.Zebra
alias App.Apple

# good
alias App.Apple
alias App.Zebra
```

Use the __MODULE__ pseudo-variable

```
# bad
%MyApp.User{name: n}

# good
%__MODULE__ {name: n}
```

Typespecs

@spec directly before the function

```
# bad
@spec f() :: :ok

def f, do: :ok

# good
@spec f() :: :ok
def f, do: :ok
```

Keep @typedoc with its @type

```
# bad
@type t :: map()
# â flaterâ f
@typedoc "a thing"

# good
@typedoc "a thing"
@type t :: map()
```

Name the main type t

```
# bad
@type user :: map()

# good
@type t :: map()
```

One part per line in long union types

```
# bad
@type t :: :aaaaa | :bbbbbb | :ccccc | :dddd | :eeee | :fffff | :ggggg | :hhhhh | :iiii | :jjjjj | :kkkkk

# good
@type t ::
  :aaaaa
  | :bbbbbb
  | :ccccc
  | :dddd
  | :eeee
  | :fffff
  | :ggggg
  | :hhhhh
  | :iiii
  | :jjjjj
  | :kkkkk
```

Parentheses on zero-arity types

```
# bad
@type t :: foo

# good
@type t :: foo()
```

Expressions

Avoid needless metaprogramming

```
# bad
defmacro add(a, b), do: quote(do: unquote(a) + unquote(b))

# good
def add(a, b), do: a + b
```


Avoid needless pipelines

```
# bad
x |> f()

# good
f(x)
```

Don't deeply nest conditionals

```
# bad
if a do
  if b, do: c
end

# good
if a and b, do: c
```

Format with/else over multiple lines

```
# bad
with {:ok, x} <- f() do
  x
else _ -> :error end

# good
with {:ok, x} <- f() do
  x
else
  _ -> :error
end
```

No always-true conditionals

```
# bad
if true do
  go()
end

# good
go()
```

No anonymous functions in pipelines

```
# bad
x |> (fn v -> v + 1 end).()

# good
x |> increment()
```

No else clause with unless

```
# bad
unless x do
  a
else
  b
end

# good
if x do
  b
else
  a
end
```

No IEx.pry in production code

```
# bad
def f(x) do
  IEx.pry()
  x
end

# good
def f(x) do
  x
end
```

No IO.inspect in production code

```
# bad
value |> IO.inspect() |> use()

# good
use(value)
```

No parentheses around anonymous function arguments

```
# bad
fn(x) -> x end

# good
fn x -> x end
```

No parentheses around if/unless conditions

```
# bad
if(ready) do
  go()
end

# good
if ready do
  go()
end
```

No redundant parentheses

```
# bad
(1 + 2)

# good
1 + 2
```

No unless with a negated condition

```
# bad
unless not ready do
  go()
end

# good
if ready do
  go()
end
```

Omit brackets from keyword lists

```
# bad
foo([a: 1])

# good
foo(a: 1)
```

Omit else that returns nil

```
# bad
if x, do: a, else: nil

# good
if x, do: a
```

Omit parentheses on zero-argument definitions

```
# bad
def go(), do: :ok

# good
def go, do: :ok
```

Parentheses around definition arguments

```
# bad
def add a, b do
  a + b
end

# good
def add(a, b) do
  a + b
end
```

Parentheses on function calls with arguments

```
# bad
rem 10, 3

# good
rem(10, 3)
```

Parentheses on piped one-arity calls

```
# bad
x |> Enum.reverse

# good
x |> Enum.reverse()
```

Parentheses on zero-arity function calls

```
# bad
DateTime.utc_now

# good
DateTime.utc_now()
```

Shorthand map syntax for atom keys

```
# bad
%{:a => 1, :b => 2}

# good
%{a: 1, b: 2}
```

Start pipe chains with a raw value

```
# bad
Repo.all(query) |> Enum.map(&f/1)

# good
query |> Repo.all() |> Enum.map(&f/1)
```

Use <> to match binaries

```
# bad
<<"foo", rest::binary>> = str

# good
"foo" <> rest = str
```

Use and/or/not for boolean checks

```
# bad
ready && ok

# good
ready and ok
```

Use do: for single-line if/unless

```
# bad
if x do
  a
end

# good
if x, do: a
```

Use keyword-list shorthand

```
# bad
[{:a, 1}, {:b, 2}]

# good
[a: 1, b: 2]
```

Use pipelines for chained calls

```
# bad
f(g(h(x)))

# good
x |> h() |> g() |> f()
```

Use true as the last cond clause

```
# bad
cond do
  x -> 1
  :else -> 2
end

# good
cond do
  x -> 1
  true -> 2
end
```

Verbose map syntax for non-atom keys

```
# bad
%{"b" => 2, a: 1}

# good
%{"b" => 2, :a => 1}
```


Documentation

@moduledoc false when undocumented

```
# bad
defmodule Internal do
end

# good
defmodule Internal do
  @moduledoc false
end
```

Always include @moduledoc

```
# bad
defmodule X do
  def f, do: 1
end

# good
defmodule X do
  @moduledoc "â f"
  def f, do: 1
end
```

Blank line after @moduledoc

```
# bad
@moduledoc "X"
def f, do: 1

# good
@moduledoc "X"

def f, do: 1
```

Use heredocs for documentation

```
# bad
@doc "line one\nline two"

# good
@doc """
line one
line two
"""
```

Comments

Avoid superfluous comments

```
# bad
# increment counter
counter = counter + 1

# good
counter = counter + 1
```

Capitalise and punctuate comments

```
# bad
# round the total

# good
# Round the total.
```

Comments only for important details

```
# bad
# add one to x
x = x + 1

# good
# round half-up for legacy invoices
x = round_up(x)
```

Limit comment lines to 100 characters

```
# bad
# one very long comment line that runs well past the hundred-character limit and keeps going

# good
# A shorter comment,
# wrapped across lines.
```

Prefer @doc/@moduledoc over comments

```
# bad
# Adds two numbers
def add(a, b), do: a + b

# good
@doc "Adds two numbers"
def add(a, b), do: a + b
```

Uppercase annotation keywords (TODO/FIXME/â€¦)

```
# bad
# fix this later

# good
# FIXME: handle the timeout case
```

Testing

Expected value on the right in assertions

```
# bad
assert 200 == status

# good
assert status == 200
```

Exceptions

Error suffix for exception modules

```
# bad
defmodule App.BadInput do
end

# good
defmodule App.BadInputError do
end
```

Lowercase exception messages

```
# bad
raise "Invalid input"

# good
raise "invalid input"
```

No trailing punctuation in exception messages

```
# bad
raise "invalid input."

# good
raise "invalid input"
```

Regex

Prefer pattern matching over regular expressions

```
# bad
Regex.match?(~r/^foo/, s)

# good
match?("foo" <> _, s)
```

Use \A and \z for string boundaries

```
# bad
~r/^foo$/

# good
~r/\Afoo\z/
```

Use non-capturing regex groups

```
# bad
~r/(foo|bar)/

# good
~r/(? :foo|bar)/
```

Use the ~r sigil for regexes

```
# bad
Regex.compile!("foo")

# good
~r/foo/
```